API DESIGN FOR MICROSERVICES

A Research Report

Submitted in partial fulfillment of the
Requirements for the award of the Degree of

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

By

Amaan Jabbar Sayyed
Roll Number: 811

Under the esteemed guidance of
Mrs. Ruchi Dubey

Designation

DEPARTMENT OF INFORMATION TECHNOLOGY
SHRI RAJASTHANI SEVA SANGH'S
SMT.PARMESHWARIDEVI DURGADUTT TIBERWALA LIONS
JUHU COLLEGE OF ARTS, COMMERCE & SCIENCE
(Affiliated to University of Mumbai)
(Autonomous)
JB NAGAR, ANDHERI EAST, MUMBAI MAHARASHTRA - 400059
2025 - 26

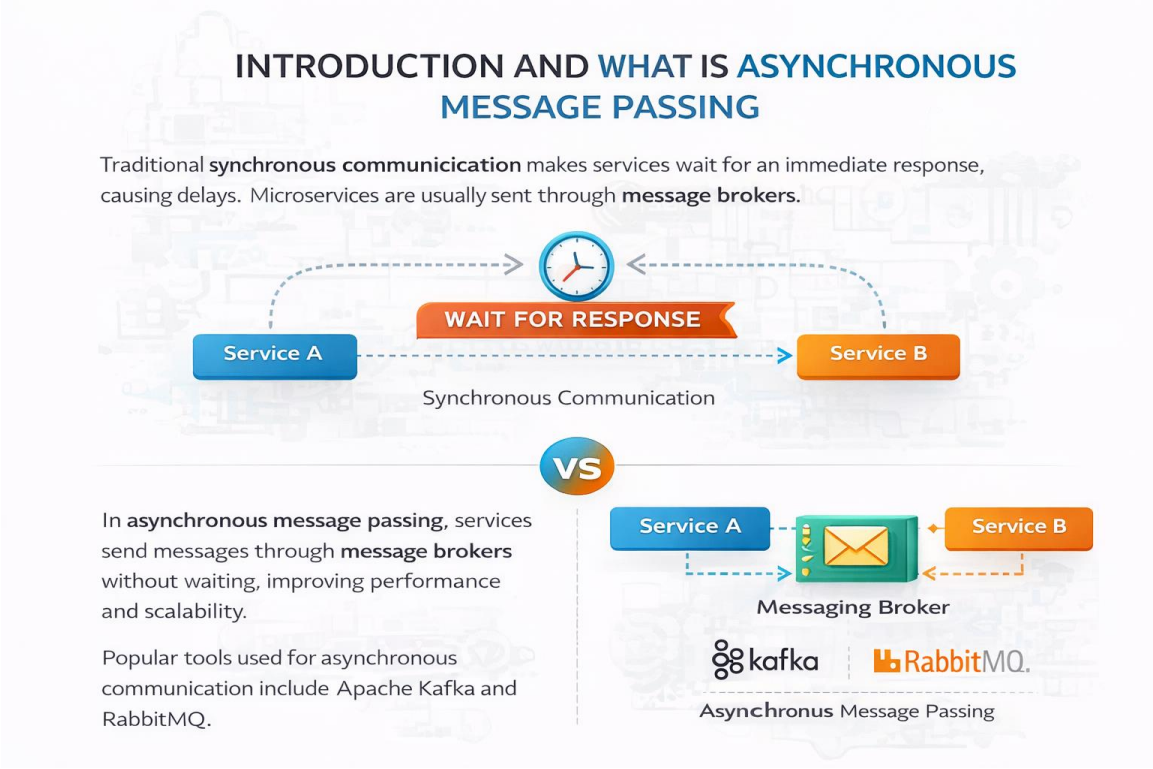
TABLE OF CONTENTS

INTRODUCTION

In microservices architecture, communication between services is essential for performing different operations. Traditional communication methods are synchronous, where one service waits for a response from another. This can cause delays and reduce system performance, especially in large applications.

To overcome this, asynchronous message passing is used. It is a communication method in which one service sends a message without waiting for an immediate response. Instead, messages are sent through a message broker, which acts as an intermediary between services.

Popular tools used for asynchronous communication include **Apache Kafka** and **RabbitMQ**. RabbitMQ works using message queues, ensuring reliable message delivery between services. Kafka, on the other hand, uses distributed topics and is designed for handling large-scale real-time data. These tools help services communicate efficiently while remaining independent.

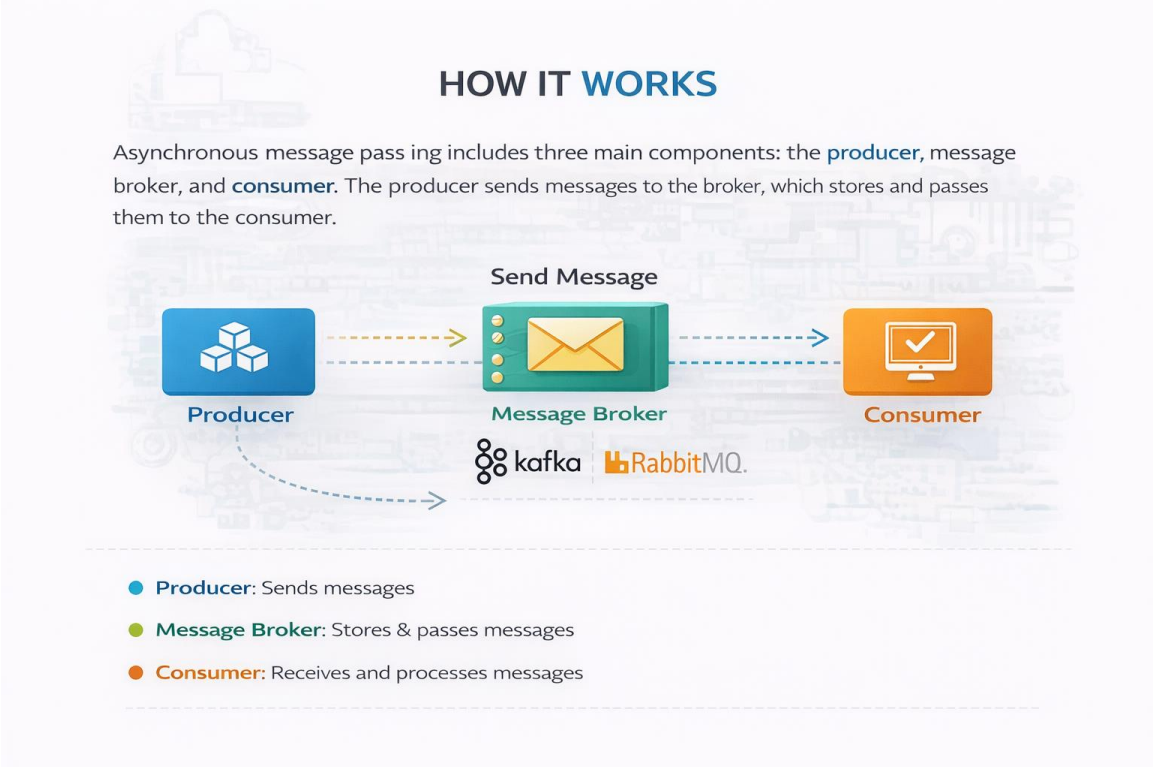


WORKING OF ASYNCHRONOUS MESSAGE PASSING TECHNIQUES

Asynchronous message passing works through three main components: producer, message broker, and consumer. The producer service sends a message to a message broker such as Kafka or RabbitMQ. The broker stores the message and ensures it is delivered to the appropriate consumer.

The consumer service receives the message and processes it when it is ready. Since the producer does not wait for the response, both services can continue working independently. In RabbitMQ, messages are stored in queues, while in Kafka, messages are stored in topics that can be consumed by multiple services.

This process improves system efficiency and allows smooth communication between microservices without direct dependency.



ADVANTAGES

Asynchronous message passing provides several benefits in microservices architecture. It improves performance by allowing services to operate independently without waiting for responses. This reduces delays and increases system speed.

It also enhances scalability, as services can process messages based on their capacity. Tools like Kafka and RabbitMQ allow handling large volumes of data efficiently. Another important advantage is fault tolerance. If a service fails, messages remain in the queue or topic and can be processed later, preventing data loss.

Additionally, this approach reduces coupling between services, making the system more flexible and easier to maintain.

CONCLUSION

Asynchronous message passing is an important communication technique in microservices architecture. It allows services to interact without depending on immediate responses, improving system performance and scalability.

In conclusion, the use of tools like Kafka and RabbitMQ makes asynchronous communication more efficient and reliable. This approach helps build flexible, scalable, and robust systems, making it a key component in modern microservices applications.